# Bases de données et pattern Singleton

Séance 2 – Support de cours et TP (3 h) – version étudiant

**Prérequis :** séance 1 (PHP 8.2, formulaires)

**Environnement :** XAMPP (PHP, MariaDB, phpMyAdmin), VSCodium

## 1 PRÉSENTATION DE LA SÉANCE

La séance 1 vous a appris à traiter un formulaire. Aujourd'hui, vos pages PHP vont **enregistrer et relire des données dans une base MariaDB** : à la fin des 3 h, vous aurez une petite application avec inscription, connexion, déconnexion et deux rôles (administrateur et utilisateur).

### Compétences visées

- Créer une base et une table, et préparer un compte de base de données réservé à l'application.
- Se connecter avec PDO et exécuter des requêtes **préparées**.
- Expliquer une injection SQL et l'empêcher.
- Écrire et justifier un **Singleton** en PHP 8.2 (`Database`, `Logger`, `SessionManager`).
- Enregistrer des mots de passe de façon sûre (`password_hash`, `password_verify`).
- Protéger des pages avec une session et contrôler l'accès selon un rôle.

### Prérequis et matériel

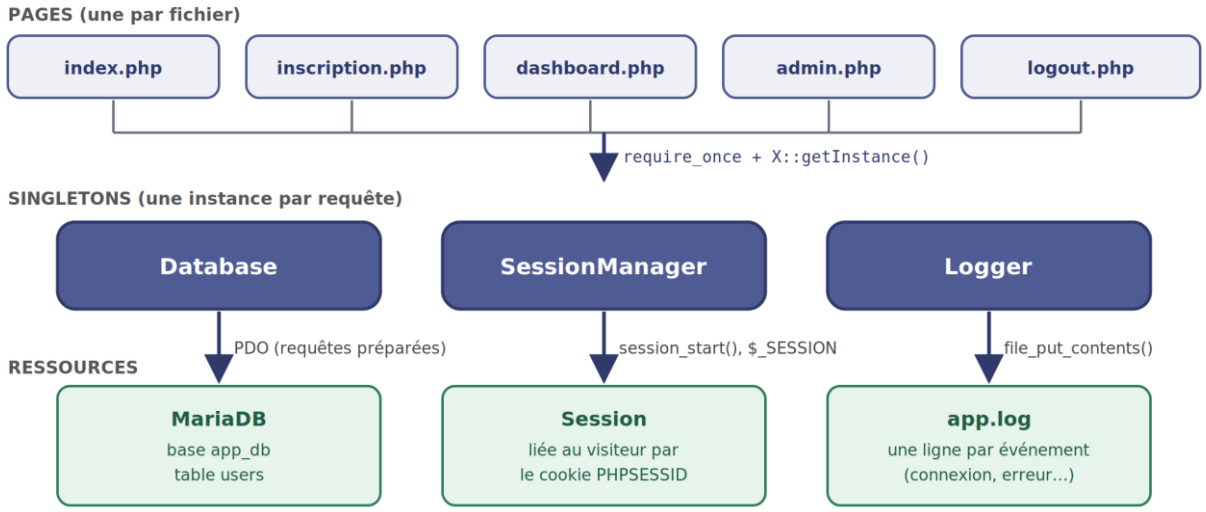
- La séance 1 : formulaires, validation, échappement avec `htmlspecialchars`, motif PRG, `declare(strict_types=1)`, `enum`, classes `readonly`.
- Quelques notions de SQL (`SELECT`, `INSERT`) : un rappel est fourni.
- Sur le poste : **XAMPP** (PHP, MariaDB et phpMyAdmin), **VSCodium** et un navigateur.

### Déroulé de la séance

| Créneau | Durée | Contenu |
|---|---|---|
| 0 h 00 – 0 h 25 | 25 min | Partie 1 : environnement MariaDB et rappels SQL. Exercice 1 |
| 0 h 25 – 1 h 00 | 35 min | Partie 2 : PDO et requêtes préparées. Exercice 2 |
| 1 h 00 – 1 h 10 | 10 min | Pause |
| 1 h 10 – 1 h 40 | 30 min | Partie 3 : le pattern Singleton. Exercice 3 |
| 1 h 40 – 2 h 15 | 35 min | Partie 4 : mots de passe, sessions et rôles. Exercice 4 |
| 2 h 15 – 2 h 55 | 40 min | Exercice 5 : inscription et rôles (mini-projet) |
| 2 h 55 – 3 h 00 | 5 min | Bilan et questions |

### 1.1 Ce que vous allez construire

Cinq pages PHP s'appuient sur trois classes Singleton, qui donnent accès à la base de données, à la session et au journal de l'application.




*Architecture de l'application : pages, Singletons et ressources*

**Arborescence du projet** (dossier `tp-bdd`)

```
tp-bdd/
├── setup.sql              création de la base et de la table
├── hash.php               outil : calcule un hash de mot de passe
├── fonctions.php          h(), champ()… (séance 1)
├── Database.php           Singleton : connexion PDO
├── Logger.php             Singleton : journal app.log
├── SessionManager.php     Singleton : session
├── Utilisateur.php        enum Role + classe readonly Utilisateur
├── index.php              connexion
├── inscription.php        création de compte
├── dashboard.php          page protégée
├── admin.php              page réservée aux administrateurs
└── logout.php             déconnexion
```

> **Conseil :** comme en séance 1, **retapez** le code des exemples. Vous le comprendrez bien mieux qu'en le copiant.

# 2 Partie 1 – Environnement MariaDB et rappels SQL

Jusqu'ici, tout disparaissait entre deux requêtes. Une **base de données** conserve les informations durablement. C'est un programme séparé de PHP (un *serveur de base de données*) auquel votre script se connecte, un peu comme le navigateur se connecte au serveur web.

XAMPP fournit ce serveur sous le nom « MySQL » ; c'est en réalité **MariaDB**, compatible avec MySQL. Il écoute sur le port 3306 de votre poste.

## 2.1 DÉMARRER LE SERVEUR DE BASE DE DONNÉES

1. Ouvrir le **panneau de contrôle XAMPP**.
2. Cliquer sur **Start** à côté de **MySQL**.
3. Pour utiliser l'interface web **phpMyAdmin**, démarrer aussi **Apache**, puis ouvrir `http://localhost/phpmyadmin`.

Votre site continue de tourner avec le serveur intégré de PHP (`php -S localhost:8000`) comme en séance 1 : Apache ne sert ici qu'à phpMyAdmin. Si Apache ne démarre pas (port 80 occupé), on peut se passer de phpMyAdmin et utiliser la ligne de commande :

```
C:\xampp\mysql\bin\mysql.exe -u root -e "source setup.sql"
```

## 2.2 RAPPEL SQL

| Besoin | Requête |
|---|---|
| Lire des lignes | `SELECT id, username FROM users WHERE username = 'alice';` |
| Ajouter une ligne | `INSERT INTO users (username, password_hash) VALUES ('bob', '...');` |
| Modifier | `UPDATE users SET role = 'admin' WHERE id = 1;` |
| Supprimer | `DELETE FROM users WHERE id = 3;` |
| Créer ou modifier une table | `CREATE TABLE ...` / `ALTER TABLE ...` |

Une **clé primaire** (`id`) identifie chaque ligne. Une contrainte **UNIQUE** empêche d'enregistrer deux fois la même valeur : la base refuse alors l'insertion, et c'est elle (et non le PHP) qui garantit l'unicité.

## 2.3 LE SCRIPT DE CRÉATION : SETUP.SQL

Ce script crée la base `app_db`, la table `users` et un compte de base de données `app_user` réservé à l'application. Ce compte a le droit de lire et d'écrire les données, mais **pas de modifier la structure** de la base : si votre application est attaquée, les dégâts possibles sont limités (principe du moindre privilège).

```
CREATE DATABASE IF NOT EXISTS app_db
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE app_db;

DROP TABLE IF EXISTS users;
CREATE TABLE users (
    id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Compte réservé à l'application : il n'a pas le droit de modifier la structure de la
base.
CREATE USER IF NOT EXISTS 'app_user'@'localhost' IDENTIFIED BY 'app_password';
GRANT SELECT, INSERT, UPDATE, DELETE ON app_db.* TO 'app_user'@'localhost';
FLUSH PRIVILEGES;
```

> Le compte `root` sans mot de passe est la configuration par défaut de XAMPP : acceptable sur un poste de TP, **jamais** sur un serveur réel. Le mot de passe `app_password` est lui aussi réservé au TP.

# 3 PARTIE 2 – ACCÉDER À LA BASE AVEC PDO

**PDO** (*PHP Data Objects*) est l'interface standard de PHP pour dialoguer avec une base de données. Le même code fonctionne, à quelques détails près, avec MariaDB, PostgreSQL ou SQLite.

## 3.1 SE CONNECTER

```php
<?php
declare(strict_types=1);

$pdo = new PDO(
    'mysql:host=localhost;dbname=app_db;charset=utf8mb4',   // DSN : où et quelle base
    'app_user',                                             // compte
    'app_password',                                         // mot de passe
    [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]
);
```

| Option | Rôle |
|---|---|
| `ERRMODE_EXCEPTION` | Une erreur SQL lève une `PDOException` au lieu de passer en silence |
| `FETCH_ASSOC` | Chaque ligne est un tableau associatif (`$ligne['username']`) |
| `EMULATE_PREPARES => false` | Les requêtes préparées sont de vraies requêtes préparées par le serveur |
| `charset=utf8mb4` (dans le DSN) | Les accents et les emojis sont correctement transmis |

## 3.2 LIRE DES DONNÉES

```php
$utilisateurs = $pdo->query('SELECT id, username FROM users ORDER BY username')->fetchAll();

foreach ($utilisateurs as $u) {
    echo h($u['username']) . '<br>';
}
```

- `query()` exécute une requête **sans paramètre** et renvoie un résultat.
- `fetchAll()` renvoie toutes les lignes ; `fetch()` renvoie la ligne suivante (ou `false` s'il n'y en a plus) ; `fetchColumn()` renvoie une seule valeur.
- Les données lues en base restent des données **non fiables** : on les échappe avec `h()` avant de les afficher, comme en séance 1.

## 3.3 L'INJECTION SQL ET LES REQUÊTES PRÉPARÉES

La tentation du débutant est de construire la requête en collant la saisie de l'utilisateur dans la chaîne SQL.

```
$sql = "SELECT id, username FROM users WHERE username = '$nom'";   // DANGEREUX
```

Si l'utilisateur saisit `'x' OR '1'='1'`, la requête devient :

```
SELECT id, username FROM users WHERE username = 'x' OR '1'='1'
```

La condition est toujours vraie : **tous** les utilisateurs sont renvoyés. Avec une autre saisie, un attaquant peut contourner une connexion, lire des tables qu'il ne devrait pas voir, voire tout supprimer. C'est l'**injection SQL**, l'une des failles les plus répandues.

La parade est la **requête préparée** : on envoie d'abord la requête avec des emplacements, puis les valeurs séparément. Le serveur sait alors que les valeurs sont des *données* et ne les interprète jamais comme du SQL.

```
$stmt = $pdo->prepare('SELECT id, username FROM users WHERE username = :nom');
$stmt->execute(['nom' => $nom]);
$resultats = $stmt->fetchAll();
```

- Les paramètres peuvent être **nommés** (`:nom`) ou **positionnels** (`?`).
- **Règle absolue : aucune variable ne doit jamais être collée dans une chaîne SQL.** Seules les valeurs passent par `execute()`.
- Les noms de tables et de colonnes ne peuvent pas être des paramètres : si vous en avez besoin d'une variable, vérifiez-le par rapport à une liste fermée écrite dans votre code (comme `tryFrom` en séance 1).

## 3.4 ÉCRIRE DES DONNÉES

```
$stmt = $pdo->prepare('INSERT INTO users (username, password_hash) VALUES (:username,
:hash)');
$stmt->execute(['username' => 'dave', 'hash' => $hash]);

$id = $pdo->lastInsertId();      // identifiant de la ligne créée
$lignes = $stmt->rowCount();     // nombre de lignes touchées (utile pour UPDATE et
DELETE)
```

## 3.5 GÉRER LES ERREURS

```
try {
    // ... requêtes ...
} catch (PDOException $e) {
    error_log('Erreur PDO : ' . $e->getMessage());     // le détail va dans le journal
    $erreur = 'Le service est momentanément indisponible.';   // l'utilisateur ne voit
rien de technique
}
```

Un message d'erreur SQL contient souvent des noms de tables ou de comptes : **ne l'affichez jamais** à l'utilisateur. On le journalise et on affiche un message neutre. Une erreur particulière est à connaître : le code SQLSTATE `23000` signale la violation d'une contrainte, par exemple une valeur `UNIQUE` déjà présente (`$e->getCode() === '23000'`).

# 4 Partie 3 – Le pattern Singleton

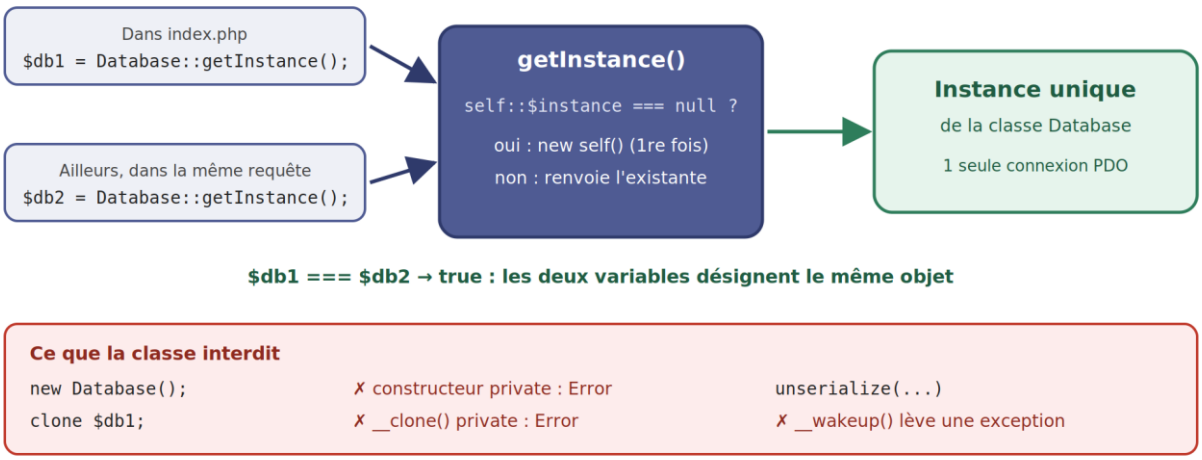
Un **patron de conception** (*design pattern*) est une solution éprouvée à un problème de programmation qui revient souvent. Le **Singleton** a deux buts :

- Garantir qu'une classe n'a **qu'une seule instance** ;
- Fournir un **point d'accès global** à cette instance.

## 4.1 Pourquoi l'utiliser ?

Les cas typiques sont ceux où plusieurs exemplaires seraient inutiles, voire dangereux :

- Une **connexion à la base de données** (dix connexions ouvertes pour une page, c'est du gaspillage) ;
- Un **journal** (*logger*) qui écrit dans un fichier ;
- Une **configuration** partagée ;
- Un **gestionnaire de session**.




*Principe du Singleton : getInstance() renvoie toujours la même instance*

## 4.2 Les étapes clés

1. Une propriété **statique** et privée stocke l'instance unique (au départ `null`).
2. Le **constructeur est privé** : impossible d'écrire `new Database()` depuis l'extérieur.
3. Une méthode **publique statique** `getInstance()` crée l'instance au premier appel, puis renvoie toujours la même.
4. On **bloque le clonage** (`__clone()` privé) et la désérialisation (`__wakeup()`), deux autres moyens de fabriquer une copie.

## 4.3 UN EXEMPLE MINIMAL

```php
<?php
declare(strict_types=1);

final class Connexion
{
    private static ?self $instance = null;

    private function __construct()
    {
        echo "Connexion établie.\n";
    }

    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    private function __clone() {}
}

$a = Connexion::getInstance();
$b = Connexion::getInstance();

if ($a === $b) {
    echo "Les deux variables contiennent la même instance.\n";
}
```

Résultat affiché : `Connexion établie.` **une seule fois**, puis `Les deux variables contiennent la même instance.`

**Décomposition de `private static ?self $instance = null;`**

| Élément | Signification |
|---|---|
| `private` | Propriété accessible uniquement depuis l'intérieur de la classe |
| `static` | Propriété qui appartient à la classe elle-même, partagée par tout le code (`self::$instance`) et accessible sans créer d'objet |
| `?self` | Type *nullable* : soit une instance de la classe (`self` désigne la classe courante), soit `null` |
| `$instance` | Nom de la propriété |
| `= null` | Valeur initiale : aucune instance n'existe encore |

Cette instance n'est créée qu'au moment où on en a besoin : c'est l'**initialisation paresseuse** (*lazy loading*), qui économise des ressources.

## 4.4 NOTRE SINGLETON DATABASE

```php
<?php
declare(strict_types=1);

final class Database
{
    private static ?self $instance = null;
    private PDO $connection;

    private function __construct()
    {
        $this->connection = new PDO(
            'mysql:host=localhost;dbname=app_db;charset=utf8mb4',
            'app_user',
            'app_password',
            [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ]
        );
    }

    public static function getInstance(): self
    {
        return self::$instance ??= new self();
    }

    public function getConnection(): PDO
    {
        return $this->connection;
    }

    private function __clone() {}

    public function __wakeup(): void
    {
        throw new LogicException('Un singleton ne peut pas être désérialisé.');
    }
}
```

Quelques choix à noter :

- `final` : personne ne peut hériter de la classe et contourner le verrouillage.
- `self::$instance ??= new self();` signifie « si la propriété est `null`, on lui affecte une nouvelle instance » : c'est la version abrégée du `if` de l'exemple précédent.
- `__wakeup()` doit rester **publique** (PHP 8 émet un avertissement si une méthode magique est privée) ; on la fait donc lever une exception pour interdire la désérialisation.
- La connexion est créée dans le constructeur : si elle échoue, `getInstance()` lève une `PDOException`, qu'il faut donc appeler **dans un bloc `try`**.

## 4.5 NOTRE SINGLETON LOGGER

```php
<?php
declare(strict_types=1);

final class Logger
{
    private static ?self $instance = null;
    private string $fichier;

    private function __construct()
    {
        $this->fichier = __DIR__ . '/app.log';
    }

    public static function getInstance(): self
    {
        return self::$instance ??= new self();
    }

    public function log(string $message): void
    {
        $message = str_replace(["\r", "\n"], ' ', $message);   // une entrée = une seule ligne
        $date    = date('Y-m-d H:i:s');
        file_put_contents($this->fichier, "[$date] $message\n", FILE_APPEND | LOCK_EX);
    }

    private function __clone() {}

    public function __wakeup(): void
    {
        throw new LogicException('Un singleton ne peut pas être désérialisé.');
    }
}
```

La ligne `str_replace(["\r", "\n"], ' ', $message)` empêche un utilisateur malveillant de fabriquer de fausses lignes de journal en glissant des retours à la ligne dans un nom d'utilisateur.

## 4.6 POINTS COMMUNS DE NOS SINGLETONS

| Élément | Rôle |
|---|---|
| Constructeur `private` | Interdit `new` depuis l'extérieur |
| Propriété statique `$instance` | Conserve l'unique instance |
| Méthode `getInstance()` | Crée l'instance au premier appel, la renvoie ensuite |
| `__clone()` privé | Interdit la copie par `clone` |
| `__wakeup()` qui lève une exception | Interdit la copie par désérialisation |

### 4.7 PRÉCAUTIONS

- **Ne pas en abuser.** Le Singleton est une variable globale déguisée : il rend les dépendances invisibles et le code plus difficile à tester (on ne peut pas facilement le remplacer par une fausse version pour un test). Réservez-le aux vraies ressources uniques.
- **En PHP, « unique » veut dire « une seule par requête ».** Rappelez-vous la séance 1 : chaque requête repart de zéro, donc chaque requête recrée ses propres instances. Un Singleton PHP n'est pas partagé entre deux visiteurs ni entre deux requêtes.
- Dans d'autres langages comme Java, l'exécution parallèle (*multithread*) oblige à protéger la création du Singleton. Avec PHP, qui exécute un script à la fois par requête, ce n'est pas un problème.

## 5 PARTIE 4 – MOTS DE PASSE, SESSIONS ET RÔLES

### 5.1 NE JAMAIS STOCKER UN MOT DE PASSE EN CLAIR

Si la base fuite, des mots de passe en clair exposent vos utilisateurs sur tous les autres sites où ils les réutilisent. On stocke donc une **empreinte** (*hash*) calculée par une fonction à sens unique.

```
$hash = password_hash('Password123', PASSWORD_DEFAULT);   // à l'inscription : on stocke $hash
$ok   = password_verify($saisie, $hash);                  // à la connexion : true ou false
```

- `PASSWORD_DEFAULT` choisit l'algorithme recommandé du moment (aujourd'hui bcrypt, qui produit 60 caractères commençant par `$2y$`). La colonne est en `VARCHAR(255)` pour pouvoir accueillir de futurs algorithmes plus longs.
- Un **sel** aléatoire est inclus dans le hash : deux appels sur le même mot de passe donnent deux hash différents, et `password_verify` sait quand même les comparer.
- Il est impossible de « décoder » un hash : on ne peut que comparer. Un mot de passe oublié se **réinitialise**, il ne se retrouve pas.

### 5.2 LE PARCOURS DE CONNEXION

1. Le formulaire envoie `username` et `password` en **POST**.
2. On cherche l'utilisateur avec une **requête préparée** (`WHERE username = :username`).
3. On vérifie le mot de passe avec `password_verify`.
4. En cas de succès : `session_regenerate_id(true)` (nouvel identifiant de session, contre la *fixation de session*), puis on enregistre l'identifiant de l'utilisateur en session.
5. **Redirection** (motif PRG) vers la page protégée.
6. En cas d'échec : **un seul message** (« Identifiants invalides. »), que l'utilisateur n'existe pas ou que le mot de passe soit faux. Deux messages différents permettraient à un attaquant de découvrir quels comptes existent.

## 5.3 NOTRE SINGLETON SESSIONMANAGER

```php
<?php
declare(strict_types=1);

final class SessionManager
{
    private static ?self $instance = null;

    private function __construct()
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    public static function getInstance(): self
    {
        return self::$instance ??= new self();
    }

    public function set(string $cle, mixed $valeur): void
    {
        $_SESSION[$cle] = $valeur;
    }

    public function get(string $cle): mixed
    {
        return $_SESSION[$cle] ?? null;
    }

    public function has(string $cle): bool
    {
        return isset($_SESSION[$cle]);
    }

    /** À appeler juste après une connexion réussie (contre la fixation de session). */
    public function regenerer(): void
    {
        session_regenerate_id(true);
    }

    public function setFlash(string $message): void
    {
        $_SESSION['flash'] = $message;
    }

    /** Lit le message puis le supprime : il ne s'affiche qu'une fois. */
    public function popFlash(): ?string
    {
        $message = $_SESSION['flash'] ?? null;
        unset($_SESSION['flash']);
        return is_string($message) ? $message : null;
    }

    public function detruire(): void
    {
        $_SESSION = [];
        session_destroy();
    }

    private function __clone() {}
```

```
    public function __wakeup(): void
    {
        throw new LogicException('Un singleton ne peut pas être désérialisé.');
    }
}
```

La méthode `popFlash()` reprend le principe du message à usage unique de la séance 1 : on lit le message puis on le supprime, pour qu'il ne s'affiche qu'une fois.

## 5.4 PROTÉGER UNE PAGE

Chaque page protégée commence par la même vérification :

```
$session = SessionManager::getInstance();

if (!$session->has('user_id')) {
    header('Location: index.php');
    exit;
}
```

## 5.5 LES RÔLES : ENUM ET CLASSE READONLY

On retrouve ici les outils de PHP 8.2 de la séance 1. Un `enum` liste les rôles possibles, une classe `readonly` transporte les données d'un utilisateur :

```
<?php
declare(strict_types=1);

enum Role: string
{
    case Admin = 'admin';
    case User  = 'user';
}

final readonly class Utilisateur
{
    public function __construct(
        public int $id,
        public string $username,
        public Role $role,
    ) {}

    /** @param array<string, mixed> $ligne une ligne de la table users */
    public static function depuisLigne(array $ligne): self
    {
        return new self(
            id: (int) $ligne['id'],
            username: (string) $ligne['username'],
            role: Role::tryFrom((string) ($ligne['role'] ?? '')) ?? Role::User,
        );
    }

    public function estAdmin(): bool
    {
        return $this->role === Role::Admin;
    }
}
```

`Role::tryFrom()` renvoie `null` pour une valeur inconnue : l'application retombe alors sur le rôle le moins privilégié (`Role::User`) au lieu de planter ou, pire, d'accorder trop de droits.

Pour réserver une page aux administrateurs :

```
$role = Role::tryFrom((string) $session->get('role')) ?? Role::User;

if ($role !== Role::Admin) {
    http_response_code(403);          // 403 : accès interdit
    echo '<h1>Accès refusé</h1>';
    exit;
}
```

> **Le contrôle d'accès se fait toujours côté serveur, sur chaque page.** Masquer un lien dans le menu n'empêche personne de taper l'adresse de la page : sans cette vérification, la page reste accessible.

# 6 TP – BASE DE DONNÉES ET SINGLETON

Ce TP comporte cinq exercices. Chacun réutilise le résultat du précédent : ne sautez pas d'étape.

## 6.1 PRÉPARATION (5 MIN)

1. Créer un dossier `tp-bdd` et l'ouvrir dans VSCodium.
2. Dans un terminal, vérifier que PHP sait parler à MariaDB : `php -m` doit lister `pdo_mysql` (sous PowerShell : `php -m | Select-String pdo_mysql`).
3. Lancer le serveur intégré depuis ce dossier : `php -S localhost:8000`, et le laisser tourner.
4. Créer `fonctions.php`, qui reprend les fonctions de la séance 1 :

```php
<?php
declare(strict_types=1);

function h(string $texte): string
{
    return htmlspecialchars($texte, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

function champ(string $cle): string
{
    $valeur = $_POST[$cle] ?? '';
    return is_string($valeur) ? trim($valeur) : '';
}

/** Les mots de passe ne sont jamais « trimés » : un espace peut en faire partie. */
function champMotDePasse(string $cle): string
{
    $valeur = $_POST[$cle] ?? '';
    return is_string($valeur) ? $valeur : '';
}

function afficherErreur(array $erreurs, string $cle): void
{
    if (isset($erreurs[$cle])) {
        echo '<div class="erreur">' . h($erreurs[$cle]) . '</div>';
    }
}
```

Comme en séance 1, chaque fichier PHP commence par `declare(strict_types=1);`, et on utilise `require_once __DIR__ . '/...'` pour inclure un autre fichier.

## 6.2 EXERCICE 1 – PRÉPARER LA BASE DE DONNÉES (20 MIN)

**Objectif :** disposer d'une base `app_db` contenant une table `users` et trois comptes de test.

1. Démarrer **MySQL** (et **Apache** pour phpMyAdmin) dans le panneau de contrôle XAMPP.
2. Créer le fichier `setup.sql` avec le contenu donné dans la partie 1.
3. L'importer : dans phpMyAdmin, onglet **Importer**, choisir le fichier puis **Importer** ; ou en ligne de commande, depuis le dossier du projet : `C:\xampp\mysql\bin\mysql.exe -u root -e "source setup.sql"`.
4. Créer `hash.php` pour calculer l'empreinte du mot de passe de test, puis l'exécuter avec `php hash.php` et copier le résultat :

```
<?php
declare(strict_types=1);

echo password_hash('Password123', PASSWORD_DEFAULT), PHP_EOL;
```

5. Dans phpMyAdmin (onglet **SQL**) ou avec `mysql.exe -u root`, insérer trois utilisateurs en collant le hash à la place de `COLLER_LE_HASH_ICI` :

```
USE app_db;
INSERT INTO users (username, password_hash) VALUES
    ('alice', 'COLLER_LE_HASH_ICI'),
    ('bob',   'COLLER_LE_HASH_ICI'),
    ('carol', 'COLLER_LE_HASH_ICI');
```

Les trois comptes ont le mot de passe `Password123` : **pour le TP uniquement**. Ne saisissez jamais un vrai mot de passe sur un site en ligne qui « calcule des hash » : vous le confieriez à un inconnu.

**À vérifier**

- ☐ `SELECT id, username, LEFT(password_hash, 7) AS debut FROM users;` renvoie trois lignes dont le `debut` vaut `$2y$10$`.
- ☐ Relancer `php hash.php` donne un hash **différent** à chaque exécution. Pourquoi, et pourquoi la vérification fonctionne-t-elle quand même ?
- ☐ Le mot de passe `Password123` n'apparaît nulle part dans la base.

## 6.3 EXERCICE 2 – PDO ET INJECTION SQL (25 MIN)

**Objectif :** lire la base avec PDO, attaquer une page vulnérable, puis la corriger.

**Partie A – Lire les utilisateurs.** Créer `test_pdo.php` : connexion PDO avec les options de la partie 2 (compte `app_user`), requête `SELECT id, username, created_at FROM users ORDER BY username`, affichage dans une liste `<ul>` (avec `h()`). L'ensemble est dans un bloc `try ... catch (PDOException $e)` : en cas d'erreur, on journalise avec `error_log()` et on affiche un message neutre.

**Partie B – Une page vulnérable.** Créer `vulnerable.php` avec le code suivant, qui cherche un utilisateur par son nom :

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/fonctions.php';

$pdo = new PDO(
    'mysql:host=localhost;dbname=app_db;charset=utf8mb4',
    'app_user',
    'app_password',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC]
);

$nom = $_GET['nom'] ?? '';
$nom = is_string($nom) ? $nom : '';
$sql = '';
$resultats = [];

if ($nom !== '') {
    $sql = "SELECT id, username FROM users WHERE username = '$nom'";   // DANGEREUX :
concaténation
    $resultats = $pdo->query($sql)->fetchAll();
}
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Recherche (version vulnérable)</title></head>
<body>
  <h1>Rechercher un utilisateur</h1>
  <form method="get">
    <input type="text" name="nom" value="<?= h($nom) ?>">
    <button type="submit">Chercher</button>
  </form>
  <?php if ($sql !== ''): ?>
    <p>Requête exécutée : <code><?= h($sql) ?></code></p>
    <ul>
      <?php foreach ($resultats as $r): ?>
        <li><?= h($r['username']) ?></li>
      <?php endforeach; ?>
    </ul>
  <?php endif; ?>
</body>
</html>
```

1. Chercher `alice`. Que montre la ligne « Requête exécutée » ?
2. Chercher `x' OR '1'='1`. Que se passe-t-il ?
3. Chercher une simple apostrophe `'`. Que voit le visiteur ? Regarder aussi le terminal du serveur.

**Partie C – Corriger.** Remplacer la concaténation par une requête préparée (paramètre `:nom`) et recommencer les trois tests. Conserver l'affichage de la requête SQL **avec** son paramètre pour comparer (`SELECT ... WHERE username = :nom`).

**À vérifier**

- ☐ `test_pdo.php` liste les trois utilisateurs.
- ☐ Avec la version vulnérable, la saisie `x' OR '1'='1` renvoie les trois utilisateurs.
- ☐ Avec la version corrigée, la même saisie ne renvoie **rien** : pourquoi ?
- ☐ Remplacer temporairement le mot de passe de la base par une valeur fausse dans `test_pdo.php` affiche un message neutre dans le navigateur, et le détail technique dans le terminal.

## 6.4 Exercice 3 – Singleton Database et Logger (15 min)

**Objectif :** centraliser l'accès à la base et au journal.

- Créer `Database.php` et `Logger.php` à partir des modèles de la partie 3 (retapez-les).
- Dans `test_pdo.php`, remplacer la création de la connexion par `Database::getInstance()->getConnection()`.
- Dans le bloc `catch`, ajouter une ligne dans le journal avec `Logger::getInstance()->log(...)`.
- Créer `singleton_test.php` qui vérifie le comportement du Singleton :
  - deux appels à `Database::getInstance()` renvoient le même objet (comparer avec `===` et afficher le résultat avec `var_dump`) ;
  - `new Database()` est refusé (afficher le message de l'`Error` capturée) ;
  - `clone` est refusé ;
  - `unserialize(serialize(Logger::getInstance()))` est refusé. (Pourquoi utiliser `Logger` et pas `Database` pour ce dernier test ?)
- Lancer ce script avec `php singleton_test.php` dans le terminal.

**À vérifier**

- ☐ `test_pdo.php` fonctionne toujours, sans connexion PDO écrite dans la page.
- ☐ Les trois tentatives de copie produisent un message d'erreur clair.
- ☐ Provoquer une panne (arrêter MySQL dans XAMPP) puis recharger la page : le visiteur voit un message neutre et `app.log` contient l'erreur.
- ☐ Question : si deux visiteurs chargent la page en même temps, y a-t-il une ou deux connexions à la base ? Pourquoi ?

## 6.5 Exercice 4 – Connexion et déconnexion (35 min)

**Objectif :** une page de connexion qui protège une page privée.

- Créer `SessionManager.php` à partir du modèle de la partie 4.
- Créer `index.php` (connexion) :
  - si l'utilisateur est déjà connecté, redirection vers `dashboard.php` ;
  - en POST : lire `username` avec `champ()` et `password` avec `champMotDePasse()` (un mot de passe n'est jamais « trimé »), chercher l'utilisateur avec une **requête préparée**, vérifier avec `password_verify` ;
  - succès : `regenerer()`, enregistrer `user_id` et `username` en session, journaliser, rediriger vers `dashboard.php` ;
  - échec : message unique « Identifiants invalides. » et journalisation de l'échec ;
  - erreur PDO : message neutre et journalisation du détail ;
  - afficher le message à usage unique (`popFlash()`) s'il existe ;
  - toutes les sorties sont échappées avec `h()`.
- Créer `dashboard.php` : redirige vers `index.php` si personne n'est connecté, sinon affiche « Bienvenue, <nom> ! », un lien de déconnexion, et journalise l'accès.
- Créer `logout.php` : journalise la déconnexion, détruit la session et redirige vers `index.php`.

**À vérifier**

- ☐ `dashboard.php` ouvert sans être connecté renvoie vers `index.php`.
- ☐ Un mauvais mot de passe et un utilisateur inexistant affichent **exactement le même** message.
- ☐ `alice` / `Password123` ouvre le tableau de bord ; après déconnexion, recharger `dashboard.php` redirige de nouveau vers `index.php`.
- ☐ `app.log` contient une ligne pour chaque connexion réussie, chaque échec, chaque accès et chaque déconnexion.
- ☐ Aucun avertissement PHP n'apparaît.

## 6.6 Exercice 5 – Inscription et rôles (40 min)

**Objectif :** permettre la création de comptes et distinguer administrateurs et utilisateurs.

1. **Modifier la base** (en tant que `root`, car `app_user` n'a pas le droit de modifier la structure) : ajouter une colonne `role` à la table `users` et faire d'`alice` un administrateur.

```
USE app_db;

ALTER TABLE users
    ADD COLUMN role ENUM('user', 'admin') NOT NULL DEFAULT 'user' AFTER
password_hash;

UPDATE users SET role = 'admin' WHERE username = 'alice';
```

2. Créer `Utilisateur.php` (enum `Role` et classe `readonly`) à partir de la partie 4.
3. Modifier `index.php` : la requête lit aussi la colonne `role` ; après une connexion réussie, construire un `Utilisateur::depuisLigne($user)` et enregistrer son rôle en session (clé `role`). Ajouter un lien vers `inscription.php`.
4. Créer `inscription.php` :
   - champs : nom d'utilisateur, mot de passe, confirmation du mot de passe ;
   - règles : nom de 3 à 30 caractères parmi lettres, chiffres et _ (`preg_match`) ; mot de passe d'au moins 8 caractères ; confirmation identique ;
   - chaque erreur s'affiche à côté de son champ, le nom saisi est réaffiché (jamais les mots de passe) ;
   - insertion par requête préparée avec `password_hash` ; **c'est la contrainte `UNIQUE` de la base** qui détecte un nom déjà pris : capturer la `PDOException` et tester le code `23000` ;
   - succès : journaliser, `setFlash('Compte créé : ...')`, rediriger vers `index.php`.
5. Modifier `dashboard.php` : afficher le rôle ; n'afficher le lien vers `admin.php` que pour un administrateur.
6. Créer `admin.php` : redirection vers `index.php` si personne n'est connecté ; **403 « Accès refusé »** (et une ligne dans le journal) si l'utilisateur n'est pas administrateur ; sinon, un tableau HTML de tous les utilisateurs (id, nom, rôle, date d'inscription).

**À vérifier**

- ☐ Créer un compte `dave` : retour sur la page de connexion avec le message de confirmation, qui ne réapparaît pas au rechargement.
- ☐ Recréer `dave` : message « déjà pris », sans exception visible.
- ☐ Dans la base, le mot de passe de `dave` est un hash `$2y$...`, jamais le texte saisi.
- ☐ `dave` (rôle `user`) tapant l'adresse `admin.php` obtient une page « Accès refusé » (code 403) ; `alice` voit la liste.
- ☐ Un nom d'utilisateur `<script>alert(1)</script>` est refusé par la validation et n'est jamais exécuté.
- ☐ `app.log` contient les inscriptions, les connexions (avec le rôle) et les accès refusés.

**Pour aller plus loin (bonus)**

- Permettre à un administrateur de changer le rôle d'un utilisateur (formulaire en POST, requête préparée `UPDATE`), sans qu'il puisse se retirer lui-même ses droits.
- Le rôle est stocké en session à la connexion : que se passe-t-il si un administrateur rétrograde un utilisateur déjà connecté ? Comment y remédier ?
- Limiter les tentatives de connexion échouées.
- Rendre la déconnexion plus sûre (elle ne devrait pas être déclenchée par un simple lien) et ajouter un jeton CSRF aux formulaires.

# 7 ANNEXES

## 7.1 AIDE-MÉMOIRE

| Besoin | Code |
|---|---|
| Se connecter | `new PDO($dsn, $user, $pass, $options)` |
| Requête sans paramètre | `$pdo->query('SELECT ...')->fetchAll()` |
| Requête avec paramètres | `$stmt = $pdo->prepare('... WHERE x = :x'); $stmt->execute(['x' => $x]);` |
| Une ligne / toutes / une valeur | `$stmt->fetch()` / `$stmt->fetchAll()` / `$stmt->fetchColumn()` |
| Id de la ligne insérée | `$pdo->lastInsertId()` |
| Nombre de lignes modifiées | `$stmt->rowCount()` |
| Contrainte violée | `$e->getCode() === '23000'` (dans un `catch (PDOException $e)`) |
| Hasher un mot de passe | `password_hash($mdp, PASSWORD_DEFAULT)` |
| Vérifier un mot de passe | `password_verify($mdp, $hash)` |
| Après connexion réussie | `session_regenerate_id(true)` |
| Page interdite | `http_response_code(403); exit;` |
| Singleton | constructeur privé, `private static ?self $instance`, `getInstance()`, `__clone()` privé |

## 7.2 B. ERREURS FRÉQUENTES ET DIAGNOSTIC

| Symptôme | Cause probable | Solution |
|---|---|---|
| `could not find driver` | Extension `pdo_mysql` désactivée | Vérifier `php -m` ; activer `extension=pdo_mysql` dans `C:\xampp\php\php.ini` |
| `SQLSTATE[HY000] [2002]` (connexion refusée) | MySQL n'est pas démarré dans XAMPP | Cliquer sur **Start** ; essayer `127.0.0.1` à la place de `localhost` dans le DSN |
| `SQLSTATE[HY000] [1045] Access denied for user 'app_user'` | Compte non créé ou mot de passe différent | Réimporter `setup.sql` |
| `SQLSTATE[HY000] [1049] Unknown database 'app_db'` | `setup.sql` n'a pas été importé | Importer le script |
| `Table 'app_db.users' doesn't exist` | Table absente | Réimporter `setup.sql` |
| `Unknown column 'role'` | `ALTER TABLE` non exécuté | Exécuter la modification de l'exercice 5 en tant que `root` |
| `Duplicate entry ... for key 'username'` | Nom déjà utilisé | Capturer la `PDOException` et tester le code `23000` |
| `Call to private Database::__construct()` | Le constructeur privé fait son travail | Utiliser `Database::getInstance()` |
| `The magic method ... must have public visibility` | `__wakeup()` déclarée privée | La déclarer `public` |
| `Class "Database" not found` | Fichier non inclus | `require_once __DIR__ . '/Database.php';` |
| « Identifiants invalides » avec le bon mot de passe | Hash mal collé (tronqué, espace en trop) | Recalculer le hash et refaire l'`INSERT` |
| `Cannot modify header information – headers already sent` | Sortie avant `header()` ou `session_start()` | Placer la logique PHP avant tout HTML (voir séance 1) |
| phpMyAdmin inaccessible | Apache non démarré (port 80 occupé) | Utiliser `mysql.exe` en ligne de commande |
| `Undefined array key "role"` | Colonne non sélectionnée ou session ancienne | Ajouter `role` au `SELECT`, puis se reconnecter |

## 7.3 POUR ALLER PLUS LOIN

- **Injection de dépendances :** passer `$pdo` en paramètre aux fonctions et aux classes plutôt que d'appeler un Singleton partout. Le code devient testable ; le Singleton peut alors n'être utilisé qu'au point d'entrée.
- **Architecture :** séparer l'accès aux données (classes « dépôt », *repository*) de l'affichage, puis découvrir le modèle MVC.
- **Sécurité :** jetons CSRF, limitation des tentatives de connexion, cookies de session sécurisés, HTTPS.
- **Base de données :** transactions, jointures, index, sauvegardes.
- **Références :** le manuel PHP (`php.net/manual/fr`, rubrique PDO) et la documentation de MariaDB (`mariadb.com/kb`).